

AUTOMATED CI/CD PIPELINE FOR JAVA WEB APPLICATIONS

Internship Project Report

Submitted in Partial Fulfillment of the Requirements for the Completion of Internship Program

Submitted By

GOWTHAM DEGALA

Organization

Cloud Vision Technologies

Internship Duration

6 Months

Year

2025

1. Project Title

Automated CI/CD Pipeline for Java Web Applications

2. Project Overview

This project demonstrates an end-to-end Continuous Integration and Continuous Deployment (CI/CD) pipeline for Java web applications. The pipeline automates source code integration, application build, deployment, containerization, and orchestration using industry-standard DevOps tools.

The workflow begins when developers push code to GitHub. Jenkins automatically fetches the latest code, builds the application using Maven, generates a WAR file and transfers it to the Ansible server. Ansible automates deployment to Apache Tomcat, builds a Docker image, pushes the image to Docker Hub, and deploys the application to Kubernetes.

3. Architecture Description

3.1 GitHub Repository

Repository Name: sampleapp

Branch: main

Stores Java web application source code

Acts as the central version control repository

3.2 Jenkins Server

Configured with a CI/CD pipeline

Pulls source code from GitHub

Builds the application using Maven

Generates a deployable WAR file

Transfers artifacts to the Ansible server

Triggers automated deployment tasks

3.3 Ansible Server

Receives build artifacts from Jenkins

Executes deployment playbooks

Automates Tomcat deployment

Builds Docker images

Pushes images to Docker Hub

Applies Kubernetes manifests

3.4 Apache Tomcat Server

Hosts the Java web application

Receives WAR files through Ansible automation

Provides application access for testing

3.5 Docker Engine

Containerizes the application

Builds Docker images using Dockerfile

Runs containers for application testing and deployment

3.6 Kubernetes Cluster

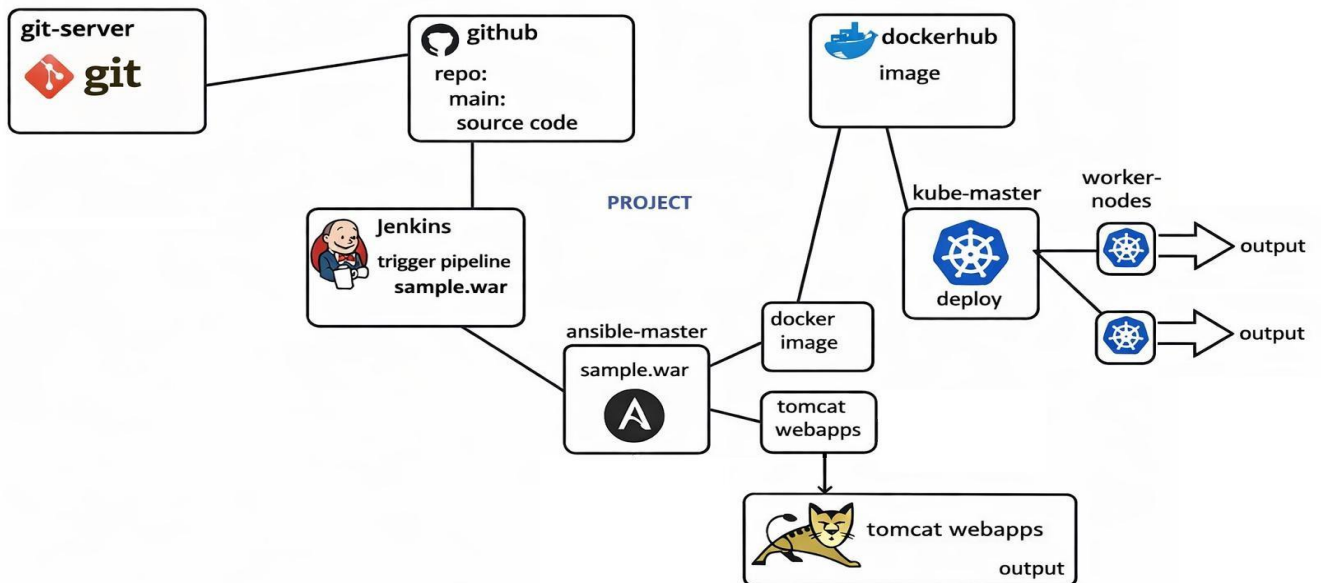
Pulls Docker images from Docker Hub

Deploys containerized applications

Manages scaling and availability

Ensures reliable application deployment

Architecture Diagram



4. Workflow

Developer pushes source code to GitHub.

Jenkins pipeline is triggered automatically.

Jenkins checks out the latest source code.

Maven builds the application and generates a WAR artifact.

Jenkins archives the build artifact.

Jenkins transfers the WAR file to the Ansible server.

Ansible deploys the application to Apache Tomcat.

Application functionality is verified using a web browser.

Ansible builds a Docker image.

Docker image is pushed to Docker Hub.

Kubernetes deploys the application using manifest files.

Kubernetes pulls the image from Docker Hub.

Kubernetes manages application deployment and scaling.

5. Jenkins Pipeline Stages

Stage 1: Source Code Checkout

Fetch latest code from GitHub repository

Stage 2: Build Application

Compile source code using Maven

Generate WAR artifact

Stage 3: Archive Artifact

Store build artifacts for future reference

Stage 4: Transfer Artifact

Send WAR file to Ansible server

Stage 5: Execute Deployment

Trigger Ansible playbook execution

Stage 6: Containerization

Build Docker image

Push image to Docker Hub

Stage 7: Kubernetes Deployment

Deploy application to Kubernetes cluster

6. Ansible Playbook Tasks

Copy WAR file to Tomcat webapps directory

Restart Tomcat service

Verify application deployment

Create Dockerfile

Build Docker image

Push Docker image to Docker Hub

Apply Kubernetes manifest files

Verify Kubernetes deployment status

7. Tools & Technologies Used

Tool	Purpose
Git	Version Control
GitHub	Source Code Repository
Jenkins	Continuous Integration & Deployment
Maven	Build Automation Tool
Ansible	Configuration Management & Deployment
Apache Tomcat	Application Server
Docker	Containerization Platform
Docker Hub	Container Registry
Kubernetes	Container Orchestration
Linux	Server Environment

8. Project Outcome

Automated the complete software delivery lifecycle.

Reduced manual deployment effort through automation.

Improved deployment consistency and reliability.

Enabled faster and more reliable release cycles.

Implemented containerized deployments using Docker and Kubernetes.

Established a scalable and repeatable CI/CD process.

9. Conclusion

This project successfully implements an end-to-end CI/CD pipeline using GitHub, Jenkins, Maven, Ansible, Apache Tomcat, Docker, and Kubernetes. The solution automates application build, testing, deployment, containerization, and orchestration, ensuring reliable and efficient software delivery with minimal manual intervention.